

CO4-AT1-Industrial Problem Solving

Ticket Booking System

1. Proposed GUI Structure

Login / User Details



Booking Window



Seat Selection Window



Confirmation Window

Booking Window

```
+-----+
|      TICKET BOOKING SYSTEM      |
+-----+
| Name:  [ _____ ]           |
| Phone: [ _____ ]           |
| Show:  [ Select Show ▼ ]        |
| Date:  [ Select Date ▼ ]        |
|                                     |
|      SEAT SELECTION             |
|                                     |
| [A1] [A2] [A3] [A4] [A5]       |
| [B1] [B2] [B3] [B4] [B5]       |
| [C1] [C2] [C3] [C4] [C5]       |
|                                     |
| [ Book Ticket ] [ Cancel ]      |
+-----+
```

2. Swing Components

Component Purpose

JFrame Creates application windows

JPanel Groups related components

Component Purpose

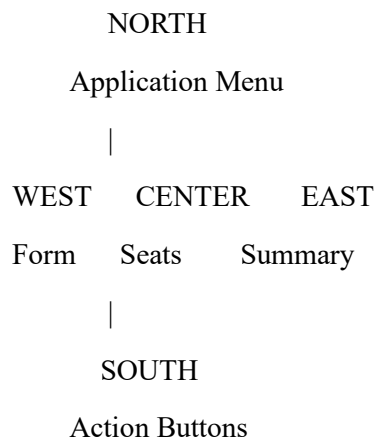
JButton	Seat selection and operations
JLabel	Displays information
TextField	Accepts user details
JComboBox	Selects show/date
JMenuBar	Provides navigation options
JOptionPane	Displays confirmation/error messages

3. Layout Managers

Different layout managers can be combined to create an organized interface.

BorderLayout

BorderLayout can be used for the main application window.



GridLayout

GridLayout is suitable for arranging seats because seats can be displayed in rows and columns.

```
JPanel seatPanel = new JPanel(new GridLayout(3, 5, 5, 5));
```

FlowLayout

FlowLayout is suitable for buttons such as Book, Cancel, and Back.

```
JPanel buttonPanel = new JPanel(new FlowLayout());
```

Using a combination of these layouts provides better flexibility than using only one layout manager.

4. Seat Selection and Event Handling

Each seat can be represented using a JButton.

```
JButton[] seats = new JButton[15];
```

```
for (int i = 0; i < seats.length; i++) {
```

```

int seatNumber = i + 1;

seats[i] = new JButton("S" + seatNumber);

seats[i].addActionListener(e -> {
    JButton selectedSeat = (JButton) e.getSource();

    if (selectedSeat.isEnabled()) {
        selectedSeat.setEnabled(false);

        JOptionPane.showMessageDialog(
            frame,
            "Seat " + selectedSeat.getText() + " selected."
        );
    }
});

seatPanel.add(seats[i]);
}

```

When a user selects a seat, the event listener identifies the selected button and updates the seat status.

5. Accurate Seat Allocation

The system must prevent two users from booking the same seat.

A simple seat-status array can be used:

```
boolean[] bookedSeats = new boolean[15];
```

When a seat is booked:

```

if (!bookedSeats[seatNumber]) {
    bookedSeats[seatNumber] = true;
} else {
    JOptionPane.showMessageDialog(
        frame,
        "Seat is already booked."
    );
}

```

```
}
```

For a real-world system, seat status should be stored in a database. This prevents duplicate bookings when multiple users access the system simultaneously.

6. Booking Validation

Before confirming a booking, the system should validate:

- Name should not be empty.
- Phone number should contain valid digits.
- A show must be selected.
- A date must be selected.
- At least one seat must be selected.

Example:

```
if (nameField.getText().trim().isEmpty()) {  
    JOptionPane.showMessageDialog(  
        frame,  
        "Please enter your name."  
    );  
    return;  
}
```

```
if (selectedSeat == null) {  
    JOptionPane.showMessageDialog(  
        frame,  
        "Please select a seat."  
    );  
    return;  
}
```

7. Exception Handling

Exception handling prevents the application from terminating unexpectedly.

```
try {  
    String phone = phoneField.getText().trim();  
  
    if (!phone.matches("\\d{10}")) {  
        throw new IllegalArgumentException(  

```

```

        "Phone number must contain 10 digits."
    );
}

JOptionPane.showMessageDialog(
    frame,
    "Input is valid."
);

} catch (IllegalArgumentException e) {
    JOptionPane.showMessageDialog(
        frame,
        e.getMessage(),
        "Input Error",
        JOptionPane.ERROR_MESSAGE
    );
}

```

8. Booking Confirmation

After successful validation, the system displays a confirmation window.

```

int choice = JOptionPane.showConfirmDialog(
    frame,
    "Confirm ticket booking?",
    "Booking Confirmation",
    JOptionPane.YES_NO_OPTION
);

if (choice == JOptionPane.YES_OPTION) {
    JOptionPane.showMessageDialog(
        frame,
        "Ticket booked successfully!"
    );
}

```

The confirmation window can display:

```
+-----+
|  BOOKING CONFIRMED  |
+-----+
| Name: Monesh        |
| Show: Movie A       |
| Date: 17-08-2026    |
| Seat: A3            |
|                    |
| Booking ID: BK1001  |
|                    |
|  [ Close ]         |
+-----+
```

9. Cancellation

The cancellation operation should first identify the booking and then ask the user for confirmation.

```
int choice = JOptionPane.showConfirmDialog(
```

```
    frame,
```

```
    "Are you sure you want to cancel?",
```

```
    "Cancel Booking",
```

```
    JOptionPane.YES_NO_OPTION
```

```
);
```

```
if (choice == JOptionPane.YES_OPTION) {
```

```
    bookedSeats[seatNumber] = false;
```

```
    JOptionPane.showMessageDialog(
```

```
        frame,
```

```
        "Booking cancelled successfully."
```

```
    );
```

```
}
```

The cancelled seat becomes available again.

10. Sample Booking Scenario

Scenario 1: Successful Booking

Input:

Name: Monesh

Phone: 9876543210

Show: Movie A

Date: 17-08-2026

Seat: A3

Process:

Enter details

↓

Select Movie A

↓

Select seat A3

↓

Click Book

↓

Confirm booking

↓

Booking Successful

Output:

Booking confirmed successfully!

Booking ID: BK1001

Seat: A3

Scenario 2: Already Booked Seat

The user selects seat A3, but the seat is already booked.

Output:

Seat A3 is already booked.

Please select another seat.

Scenario 3: Invalid Input

The user leaves the name field empty.

Output:

Please enter your name.

Scenario 4: Cancellation

The user selects booking BK1001 and clicks Cancel.

Output:

Booking BK1001 cancelled successfully.

Seat A3 is now available.

CODE:

```
import javax.swing.*;
import java.awt.*;

public class Main extends JFrame {

    JTextField nameField, phoneField;
    JComboBox<String> showBox, dateBox;
    JButton[] seats = new JButton[15];
    boolean[] bookedSeats = new boolean[15];
    int selectedSeat = -1;

    public Main() {

        setTitle("Ticket Booking System");
        setSize(600, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JMenuBar menuBar = new JMenuBar();
        JMenu menu = new JMenu("Menu");
        JMenuItem exitItem = new JMenuItem("Exit");
        exitItem.addActionListener(e -> System.exit(0));
        menu.add(exitItem);
        menuBar.add(menu);
        setJMenuBar(menuBar);

        JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
        JPanel detailsPanel = new JPanel(new GridLayout(4, 2, 5, 5));
        detailsPanel.add(new JLabel("Name:"));
        nameField = new JTextField();
```



```

detailsPanel.add(nameField);
detailsPanel.add(new JLabel("Phone:"));
phoneField = new JTextField();
detailsPanel.add(phoneField);
detailsPanel.add(new JLabel("Show:"));
showBox = new JComboBox<>(
    new String[] {"Movie A", "Movie B", "Movie C"}
);
detailsPanel.add(showBox);
detailsPanel.add(new JLabel("Date:"));
dateBox = new JComboBox<>(
    new String[] {"17-08-2026", "18-08-2026", "19-08-2026"}
);
detailsPanel.add(dateBox);
mainPanel.add(detailsPanel, BorderLayout.NORTH);
JPanel seatPanel = new JPanel(
    new GridLayout(3, 5, 10, 10)
);
seatPanel.setBorder(
    BorderFactory.createTitledBorder("Select Seat")
);
for (int i = 0; i < 15; i++) {
    final int seatNumber = i;
    seats[i] = new JButton("S" + (i + 1));
    seats[i].addActionListener(e -> {
        if (bookedSeats[seatNumber]) {
            JOptionPane.showMessageDialog(
                this,
                "Seat is already booked!");
            return;
        }
        selectedSeat = seatNumber;
    });
}

```

```

        JOptionPane.showMessageDialog(
            this,
            "Seat S" + (seatNumber + 1) + " selected."
        );
    });
    seatPanel.add(seats[i]);
}
mainPanel.add(seatPanel, BorderLayout.CENTER);
JPanel buttonPanel = new JPanel(new FlowLayout());
JButton bookButton = new JButton("Book Ticket");
JButton cancelButton = new JButton("Cancel Booking");
JButton clearButton = new JButton("Clear");
buttonPanel.add(bookButton);
buttonPanel.add(cancelButton);
buttonPanel.add(clearButton);
mainPanel.add(buttonPanel, BorderLayout.SOUTH);
bookButton.addActionListener(e -> bookTicket());
cancelButton.addActionListener(e -> cancelBooking());
clearButton.addActionListener(e -> clearFields());
add(mainPanel);
setVisible(true);
}

private void bookTicket() {
    try {
        String name = nameField.getText().trim();
        String phone = phoneField.getText().trim();
        if (name.isEmpty()) {
            throw new IllegalArgumentException(
                "Please enter your name." );}
        if (!phone.matches("\\d{10}")) {
            throw new IllegalArgumentException(
                "Phone number must contain 10 digits."
            );
        }
    }
}

```

```

    );
}
if (selectedSeat == -1) {
    throw new IllegalArgumentException(
        "Please select a seat."
    );
}
if (bookedSeats[selectedSeat]) {
    throw new IllegalArgumentException(
        "This seat is already booked."
    );
}
String show = showBox.getSelectedItem().toString();
String date = dateBox.getSelectedItem().toString();
int choice = JOptionPane.showConfirmDialog(
    this,
    "Name: " + name +
    "\nPhone: " + phone +
    "\nShow: " + show +
    "\nDate: " + date +
    "\nSeat: S" + (selectedSeat + 1) +
    "\n\nConfirm booking?",
    "Booking Confirmation",
    JOptionPane.YES_NO_OPTION
);
if (choice == JOptionPane.YES_OPTION) {
    bookedSeats[selectedSeat] = true;
    seats[selectedSeat].setEnabled(false);
    JOptionPane.showMessageDialog(
        this,
        "Ticket booked successfully!\n" +
        "Booking ID: BK100" +

```

```

        (selectedSeat + 1)
    );
}
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(
        this,
        ex.getMessage(),
        "Input Error",
        JOptionPane.ERROR_MESSAGE
    );
}
}
private void cancelBooking() {
    if (selectedSeat == -1) {
        JOptionPane.showMessageDialog(
            this,
            "Please select a seat."
        );
        return;
    }
    if (!bookedSeats[selectedSeat]) {
        JOptionPane.showMessageDialog(
            this,
            "This seat is not booked."
        );
        return;
    }
    int choice = JOptionPane.showConfirmDialog(
        this,
        "Cancel booking for seat S" +
        (selectedSeat + 1) + "?",
        "Cancel Booking",

```

```

        JOptionPane.YES_NO_OPTION
    );
    if (choice == JOptionPane.YES_OPTION) {
        bookedSeats[selectedSeat] = false;
        seats[selectedSeat].setEnabled(true);
        JOptionPane.showMessageDialog(
            this,
            "Booking cancelled successfully.\n" +
            "Seat S" + (selectedSeat + 1) +
            " is now available."
        );
        selectedSeat = -1;
    }
}

private void clearFields() {
    nameField.setText("");
    phoneField.setText("");
    showBox.setSelectedIndex(0);
    dateBox.setSelectedIndex(0);
    selectedSeat = -1;
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new Main();
    });
}
}

```